

详细设计：从体系结构到代码

软件工程与计算 II · 详细设计模块 · 第 1 讲

南京大学 · 刘钦

今日学习路径

① 详细设计基础 → ② 面向对象详细设计 → ③ GRASP模式 → ④ 控制风格 → ⑤ 模块化与信息隐藏 → ⑥ 集成测试 → ⑦ AI辅助



输入/输出/上下文

职责与协作

5大职责模式

集中/委托/分散

Parnas思想

Mock对象

LLM辅助详设

今日核心问题

1. 详细设计的**输入**是什么？**输出**又是什么？它在 SDLC 中的位置？
2. 面向对象详细设计如何通过**职责**和**协作**建立静态/动态模型？
3. 如何用 **GRASP 模式**指导职责分配？如何选择合适的**控制风格**？

预习测试（5 分钟）

请在纸上完成以下两题：

1. 面向对象方法下，详细设计的过程包括哪些主要步骤？
2. 现在要写一个 **Sink-the-Dot-Com** 游戏，其中"棋盘 Board"和"Dot-Com 战舰"两个对象，各应该由哪个类负责创建？说明理由。

提示：回顾第 3 讲的体系结构设计；想想"谁拥有创建所需的数据"。

PART 1 · 详细设计基础

输入 / 输出 / 上下文：详细设计在 SDLC 中的位置

什么是详细设计 (Detail Design)

体系结构 vs 详细设计

- 软件体系结构定义了**模块的规格**（部件、连接件、配置）
- 详细设计用**详细设计机制**实现模块
 - **中层**：（子模块化）→ OO → 类的规格
 - **低层**：DS + ALG → 类的实现

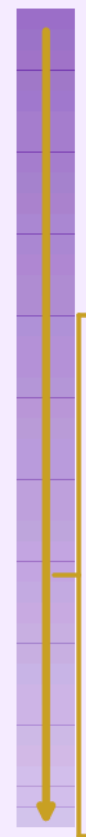
详细设计的关注点

- **美学、功能性**及模块的多个方面
- **质量属性**：
 - **Modification**（可修改）
 - **Maintenance**（可维护）
 - **Performance**（性能）
 -

详细设计 = **中层设计**（过程、调用；类、协作） + **低层设计**（数据结构、算法、类型、语句、控制结构）。

设计的抽象层次

抽象



实现

体系结构设计

高层设计
部件 · 连接件 · 配置

详细设计
中层

模块 · 接口规格
过程 · 调用 · 类 · 协作

详细设计
低层

数据结构 · 算法
类型 · 语句 · 控制结构

体系结构设计 = 高抽象，定义"是什么"；详细设计 = 趋向实现，回答"怎么做"

- 体系结构设计回答："系统有哪些大模块？它们怎么连接？"

设计思维 (Design Thinking)

Design is a form of problem-solving. Software design particularly faces **Wicked Problems** — problems with no definitive solution, only better or worse trade-offs.

五步设计过程 (Ross, Goodenough & Irvine)

步骤	核心动作	软件设计产物
1 · Crystallize	明确目的与目标	功能需求 + 质量属性 (NFR)
2 · Formulate	构想实现概念	体系结构风格 / 设计模式选择
3 · Devise	设计实现机制	类图 + 顺序图 + 算法选择
4 · Introduce	引入表达符号	UML / 伪代码 / 接口定义语言 (IDL)
5 · Describe	描述具体使用场景	设计文档 + 设计理由 (Rationale)

这五步在**体系结构设计**和**详细设计**两个阶段都会反复出现 —— 详细设计就是把体系结构决策**递归精化**到可直接编码的粒度。

思考：什么影响详细设计？

RE 的工作

- 哪些会影响详细设计？
- 用例 → 控制器 / 操作
- 领域模型 → 实体类
- 顺序图 → 类之间的协作
- 状态图 → 状态机实现
- NFR → 算法选择、并发策略

软件体系结构的工作

- 哪些会影响详细设计？
- 风格 → 类的分布方式
- 模块规格 → 类的规格上限
- 接口规范 → PO / VO / DTO
- 部署 → 远程调用、序列化
- 集成策略 → 模块间依赖方向

结论：详细设计是 RE 与体系结构在"类粒度"上的具体化与综合。

详细设计的输入

来自需求工程 (RE)

- 用例 (Use Case) —— 功能场景
- 领域模型 / 概念类图 —— 业务实体
- 系统顺序图 —— 系统级交互
- 状态图 —— 复杂对象生命周期
- 非功能需求 —— 性能、可靠性、安全等

来自体系结构

- 模块的规格 (Specification)
- 导入/导出接口 (Export / Import)
- 风格约束 (分层、MVC、微服务.....)
- 构件间接口 (如 SalesBLService、SalesDataService)

桥接类比：需求规格说明 ("在两座山间建一座桥，5 分钟内 100km/h 通过") + 体系结构 ("斜拉桥风格：桥墩、桥身、悬索") → 详细设计 ("悬索：拉索的根数；路面：路面材质")。

详细设计的上下文：从何处开始？

- Context (上下文)
 - Specification of modules: Export / Import interfaces (接口规格)
- Responsibility assignment (职责分配)
 - Some responsibilities are from RE (SRS)
 - Typically use cases, domain models, sequence diagrams, state diagrams
 - Some others are from implementation decisions

核心问题：详细设计的本质，就是把"系统的总体职责"逐步分配到具体的类与方法上 —— 既要忠于需求，又要做出合理的实现决策。

详细设计的输出

视角	产出物	UML 表示
静态结构	模块内部的类、接口与关系	类图、包图
动态行为	类之间的协作与消息传递	顺序图、通讯图
状态行为	复杂对象的生命周期	状态图
实现注解	构造方法、枚举、常量、静态方法说明	文字 + 代码片段
质量论证	为何这样设计：耦合/内聚/扩展性	设计理由（Rationale）

详细设计输出≠"画几张图"，而是为后继开发者提供**足够、清晰、可论证**的实现蓝图。

PART 2 · 面向对象详细设计

职责 (Responsibility) 与 协作 (Collaboration)

面向对象详细设计的思想

两大核心概念

- 职责 (Responsibility)
 - 类要做什么、维护什么
- 协作 (Collaboration)
 - 类之间如何通过消息一起完成大职责

设计过程

- 设计模型建立
 - 通过职责建立静态设计模型
 - 通过协作建立动态设计模型
- 设计模型重构
 - 模块化 (高内聚低耦合)
 - 信息隐藏 (隐藏职责与变更)
 - 设计模式

Rebecca Wirfs-Brock: "The objects within a program must collaborate; otherwise, the program would consist of only one big object that does everything."

Responsibilities (职责)

- A responsibility is an obligation to perform a **task** (operational responsibility) or to maintain some **data** (data responsibility) .
 - **Operational responsibilities** are usually fulfilled by **operations** (方法) .
 - **Data responsibilities** are usually fulfilled by **attributes** (属性) .
 - **Class collaborations** may be involved.

Responsibility-Driven Decomposition (职责驱动分解)

- Responsibilities may be stated at **different levels of abstraction**.
- High-level responsibilities → high-level components; 可逐层分解。
- **职责驱动的分解可能不同于功能分解** —— 职责同时反映"操作"与"数据"两方面的义务。

Responsibility Heuristics & Delegation

职责分配启发式

- 分配良好的职责 → 高内聚、低耦合
- 模块之间的职责不重叠
- 把操作和数据放在某个模块里，仅当它们帮助该模块履行其职责

Delegation（委托）

- Delegation is a tactic wherein one module (the delegator) entrusts another module (the delegate) with a responsibility.
- 一个对象可以保留高层职责，把具体执行委托给更专门的对象
- 委托 = 解耦 + 关注点分离

协作 (Collaboration): 本质

- An application can be broken down into a set of many different behaviors.
- 每一个行为由一组对象之间的协作实现
- 协作是对象网络中沿关系流动的消息模式，为完成某个行为而存在
- 协作分布在对象网络中，不存在于任何单一位置

Grady Booch (1992): "Equally important [as inheritance] is the invention of societies of objects that responsibly collaborate with one another. These societies form what I call the mechanisms of a system, and thus represent strategic architectural decisions because they transcend individual classes."

面向对象详细设计的过程

设计模型建立

- └─ 通过职责建立静态设计模型
 - └─ 抽象类的职责（数据职责 + 行为职责）
 - └─ 抽象类之间的关系（关联/聚合/组合；继承/实现）
 - └─ 添加辅助类（DTO、Helper、Util...）
- └─ 通过协作建立动态设计模型
 - └─ 抽象对象之间协作（顺序图 / 通讯图）
 - └─ 明确对象的创建（GRASP-Creator）
 - └─ 选择合适的控制风格（集中 / 委托 / 分散）

设计模型重构

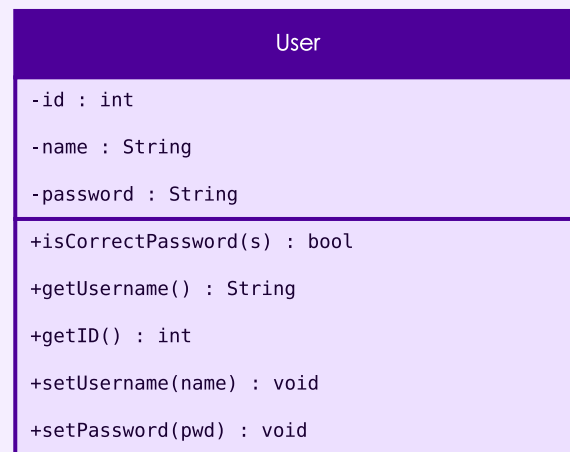
- └─ 根据模块化的思想进行重构（目标：高内聚、低耦合）
- └─ 根据信息隐藏的思想进行重构（目标：隐藏职责与变更）
- └─ 利用设计模式重构（GoF 23 / DDD / ...）

静态模型 · 抽象类的职责

类是什么？

- 类表达了对对象族本质特征的抽象
- 类是构建的蓝图
- 类承载职责：
 - 数据职责 → 属性
 - 行为职责 → 方法

单一类图示例



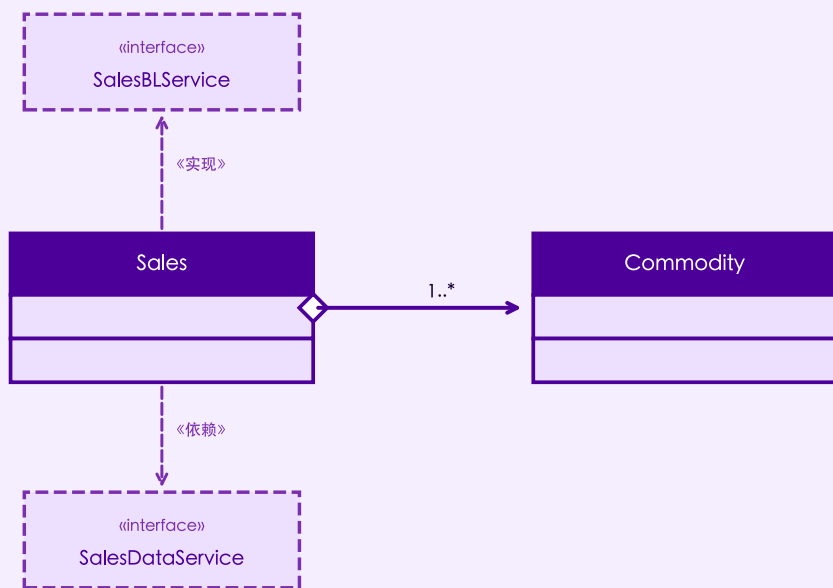
静态模型 · 类之间的关系

类型	关系	短语	解释	多重性	UML
对象级	普通关联	A has a B	A 长期持有 B 引用	A: 0..* B: 0..*	——▶
对象级	聚合	A owns B	集合所属，被聚合对象可共享	A:0..1 B:0..*	——◇
对象级	组合	B is a part of A	整体存在则部分必存在，生命周期相同	A:0..1 B:1..1	——◆
类级	继承	B is A	强关系，子类继承父类接口与实现	—	——▷
类级	实现	B implements A	类实现接口，必须实现所有方法	—	---▷

选择关系时优先考虑**生命周期**和**共享性**：能用关联就不用聚合，能用聚合就不用组合。

静态模型 · 完整类图示例

案例：销售业务



设计要点

- 业务逻辑层包 **pkg**
- **Sales** 实现接口 **SalesBLService**（被表现层调用）
- **Sales** 依赖接口 **SalesDataService**（调用数据层）
- **Sales** 聚合多个 **Commodity**
- **关键：双向依赖接口** —— 上下都通过接口解耦，便于替换与测试

动态模型 · 顺序图 (Sequence Diagram)

用途

- 表示对象之间的**协作**
- 顺序图是**交互图**的一种
- 表达对象之间**如何通过消息的传递**来完成比较大的职责

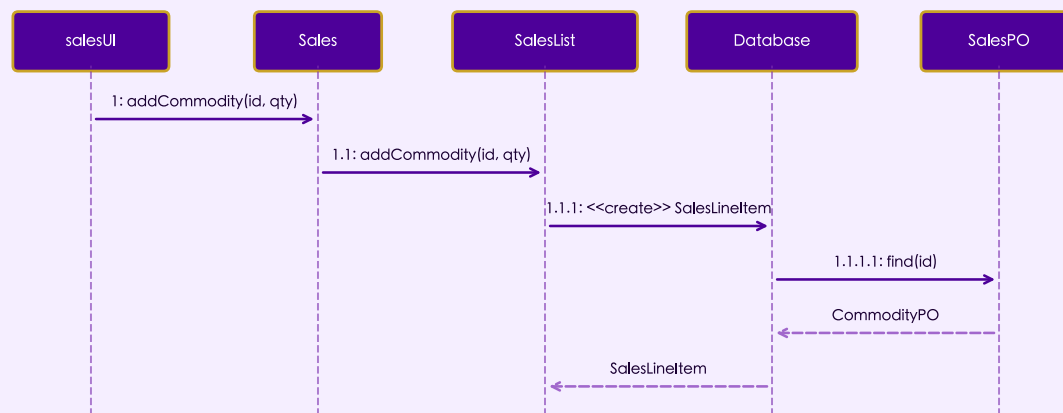
示例消息编号

1: addCommodity(id, qty)

1.1: addCommodity(id, qty)

1.1.1: <<create>> CreateMessage()

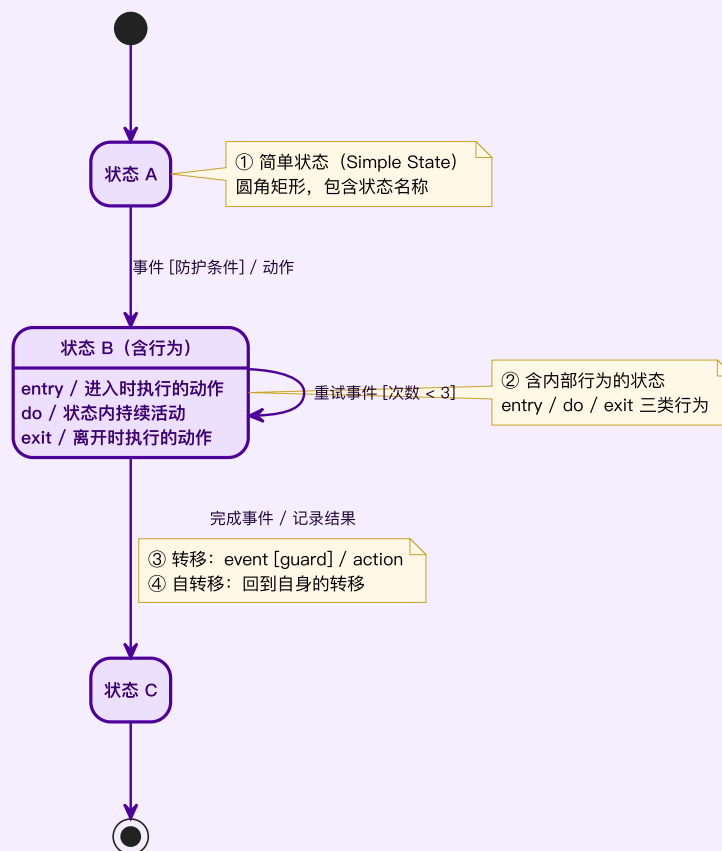
1.1.1.1: CommodityPO find(id)



动态模型 · 状态图 ① — 概念与符号

描述对象在生命周期内经历的**离散状态**及事件驱动下的**状态转移**。

要素	说明
状态 (State)	对象满足某条件、执行某活动的时期
转移 (Transition)	event [guard] / action
事件 (Event)	触发转移的信号
防护条件 (Guard)	允许转移的布尔前提 [cond]
动作 (Action)	进入/退出/转移时执行的操作
entry / do / exit	状态的三种内部行为

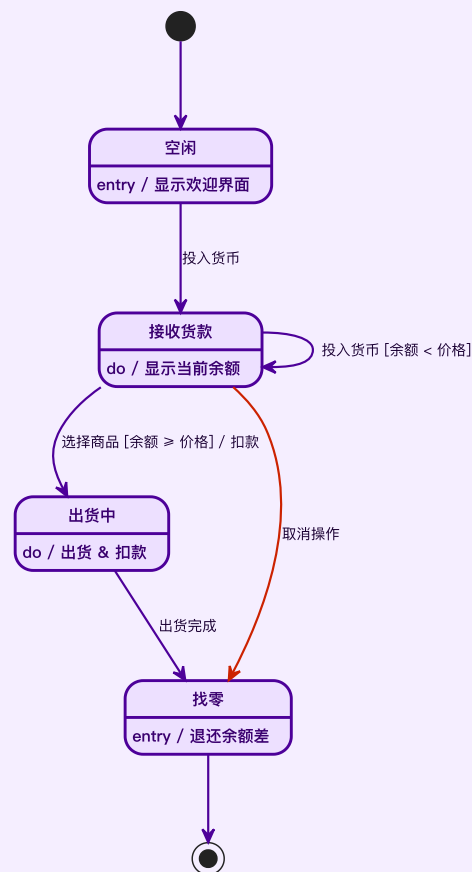


动态模型 · 状态图 ② — 例图：自动售货机

状态	entry / do
空闲	entry / 显示欢迎界面
接收货款	do / 显示余额
出货中	do / 出货 & 扣款
找零	entry / 退还余额差

关键设计点

- 投入货币 [余额 < 价格] → **自转移**，继续累计
- 选择商品 [余额 ≥ 价格] / 扣款 → **防护条件**
- 取消操作（红色）→ 直接进入找零



PART 3 · GRASP 模式

通用职责分配软件模式 · Craig Larman

GRASP Patterns

- **GRASP** = **G**eneral **R**esponsibility **A**ssignment **S**oftware **P**atterns
- Not 'design patterns', rather **fundamental principles** of object design
- Focus on one of the most important aspects of object design: **assigning responsibilities to classes**

5 个核心模式（本讲覆盖）

#	模式	解决的问题
1	Low Coupling	降低模块之间的耦合
2	High Cohesion	保持模块职责的聚焦
3	Information Expert	把职责分配给"拥有信息"的类
4	Creator	由谁负责创建对象
5	Controller	由谁负责处理系统事件

Larman 著《UML 和模式应用》是 OO 详设的经典参考。

Rule of Thumb (设计抉择经验法则)

When there are alternative design choices, take a closer look at the **cohesion and coupling implications** of the alternatives and possibly the **future evolution pressures** on the alternatives. Choose an alternative with good **cohesion, coupling and stability**.

- 不存在"唯一正确"的设计 —— 只存在**多个候选**之间的权衡
- **耦合、内聚、稳定性**是三大评价标尺
- **未来演化压力**: 考虑"哪个方案更经得起变更"
- GRASP 给出的不是公式, 而是**讨论候选方案的框架**

① Low Coupling & ② High Cohesion

Low Coupling (低耦合)

- **Problem**: 如何减少变更的影响? 如何支持低依赖、提高复用?
- **Solution**: 分配职责时保持低耦合
 - 若 B 已耦合于 A, 则把新职责也给 A, 不引入额外依赖
 - 优先依赖接口而非具体类 (→ P4)

Effect: 某类的变更对其他类影响极小; 类更易独立理解与复用。

High Cohesion (高内聚)

- **Problem**: 如何让对象保持聚焦、易理解、易管理?
- **Solution**: 分配职责时保持高内聚
 - 一个类只承担**紧密相关**的一组职责
 - 不要为了复用而拼凑不相关的操作

Effect: 类的职责清晰, 维护时改动范围小, 自然导向低耦合。

Low Coupling 与 High Cohesion 是 GRASP 的两个**元原则**—— Information Expert、Creator、Controller 的取舍, 最终都以"是否提升内聚 / 降低耦合"为评判标准。

③ Information Expert (信息专家)

- **Problem:** What is the most basic principle by which responsibilities are assigned in object-oriented design?
- **Solution:** Assign a responsibility to the class that has the information necessary to fulfill the responsibility.

案例

要计算 `Sales.total()` :

- 谁知道每个 `SalesLineItem` 的数量? → `SalesLineItem`
- 谁知道每个 `Commodity` 的价格? → `Commodity`
- → `Sales.total()` 委托给 `SalesLineItem.subtotal()`, 后者再调 `Commodity.getPrice()`

"谁有数据, 谁负责" —— 自然形成职责链, 避免数据外泄。

④ Creator (创建者)

- **Q:** Who has responsibility to create an object?
- **A:** By Creator, assign class **B** responsibility of creating instance of class **A** if (满足越多, 优先级越高):
 - **B aggregates A objects**
 - **B contains A objects**
 - **B records instances of A objects**
 - **B closely uses A objects**
 - **B has the initializing data for creating A objects**
- where there is a choice, **prefer B aggregates or contains A objects.**

案例: Who is responsible for creating `SalesLineItem` objects? → **Sale** (because Sale **contains** `SalesLineItem`, 组合关系)。

Creator · 优先级表

优先级	场景	创建地点	创建时机	备注
高	唯一属于某个整体的密不可分的一部分 (组合关系)	整体对象的属性定义和构造方法	整体对象的创建	销售业务逻辑对象由销售页面对象创建
↑	被某一对象记录和管理 (单向被关联)	关联对象的方法	业务方法执行中对象的生命周期起始点	连接池管理对象负责创建连接池对象
↓	创建所需的数据被某个对象所持有	持有数据的对象的业务方法	业务方法执行中	不知创建时机时可由别的对象创建并初始化
低	被某个整体包含 (聚合关系)	整体对象的业务方法 (非构造方法)	业务方法执行中	多个聚合时考虑高内聚低耦合决定

优势： Promotes low coupling by making instances of a class responsible for creating objects they need to reference. By creating the objects themselves, they avoid being dependent on another class to create the object for them.

⑤ Controller (控制器)

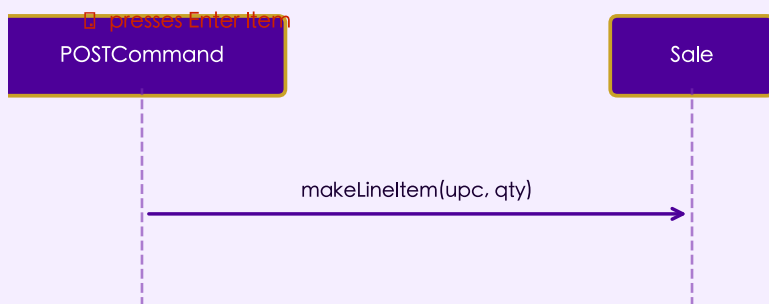
- **Problem**: How to assign responsibility for handling a system event?
- **Solution**: If a program receives events from external sources other than its graphical interface, add an event class to **decouple the event source(s) from the objects that actually handle the events**.

4 种可选的控制器

1. **The business or overall organization** (façade controller) — e.g. `Store`
2. **The overall "system"** (façade controller) — e.g. `POST`
3. **An animate thing in the domain** that would perform the work (role controller) — e.g. `Cashier`
4. **An artificial class** (Pure Fabrication) representing the use case (use case controller) — e.g. `BuyItemsHandler`

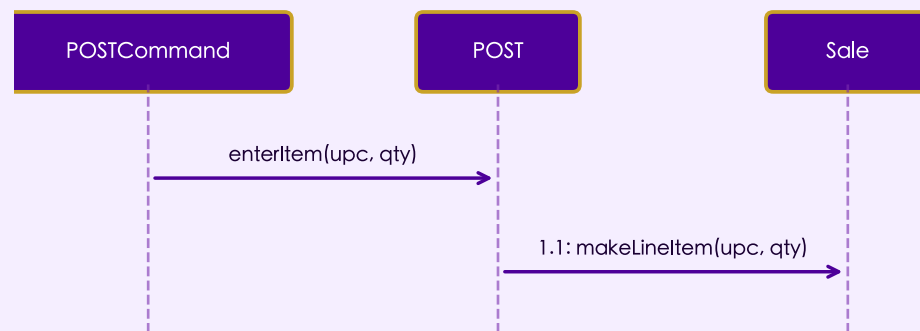
Controller · Bad Design vs Good Design

反例：Presentation 直接调 Domain



问题：表现层 POSTCommand 直接调用领域对象 Sale。业务逻辑泄漏到表现层，不可复用 / 不可测试 / 难以维护。

正例：通过 Controller 解耦



优势：表现层只发系统事件消息，由 Controller (POST) 负责协调领域对象。表现层与领域层独立演化。

PART 4 · 控制风格

集中式 / 委托式 / 分散式：系统行为如何在对象网络中分布

Collaboration (Control) Styles

A control style is a way that all system behavior is distributed among objects (components) network.

风格	特征	控制复杂度
Centralized (集中式)	A few controllers record logics of all system behaviors	控制器高复杂度，其他对象低复杂度
Delegated (委托式)	Decision making is distributed through the object networks with a few controllers making the main decisions	中复杂度对象有几个，其他低复杂度
Dispersed (分散式)	All system behavior is spread widely through the objects network	所有对象都低复杂度，但交互密集

Centralized Control (集中式控制)

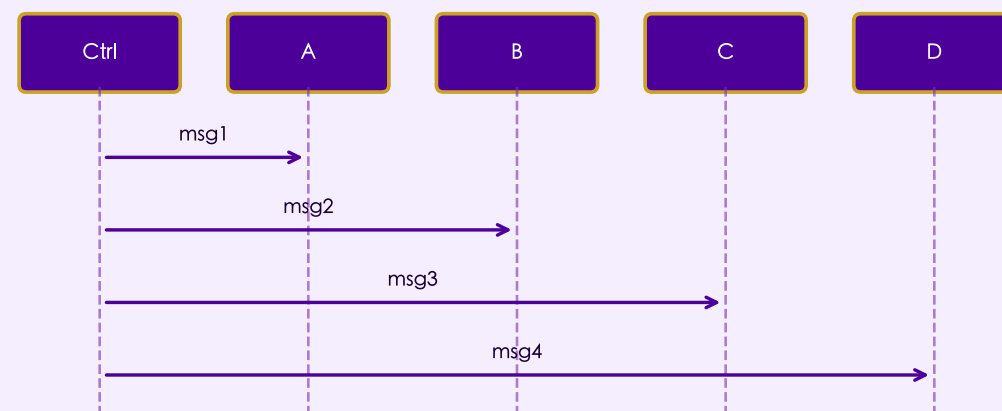
优点

- **Easy to find** where decisions are made
- **Easy to see** how decisions are made and to alter the decision-making process

缺点

- Controllers may become **bloated** — large, complex, and hard to understand / maintain / test
- Controller may treat other components as **data repositories**
 - **Increases coupling**
 - **Destroys information hiding**

集中式形态 (顺序图)



控制器统一发起所有调用，呈"星形"。

Delegated Control（委托式控制）

思想

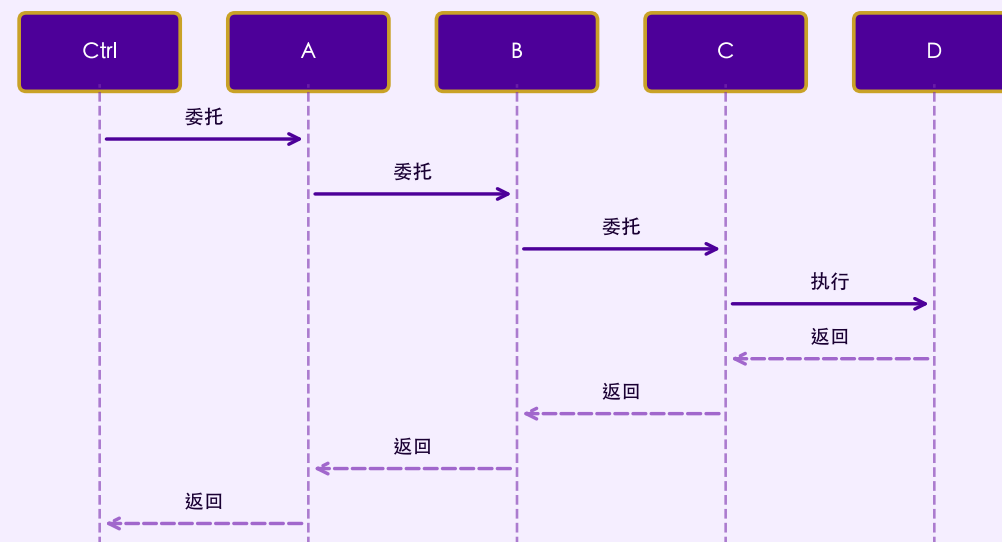
- 控制器**保留主要决策**，把次要的子任务**委托**给更专门的对象
- 形成"控制器→中间协调者→执行者"的链条
- **平衡了"集中"的清晰性和"分散"的灵活性**
- 是大多数实际系统采用的折中方案

案例：业务逻辑层

LoginController → User

SalesController → Sales、Commodity、Member

委托式形态（顺序图）



Dispersed Control (分散式控制)

- Characterized by having many components holding little data and having few responsibilities.
- It is hard to understand the flow of control.
- Components are unable to do much on their own, increasing coupling.
- It is hard to hide information.
- Cohesion is usually poor.
- Few modularity principles can be satisfied.

Heuristic (启发式经验): Avoid interactions that require each component to send many messages —— 分散式控制几乎在所有情况下都应该避免。

控制风格的选择

场景	推荐风格	理由
用例步骤清晰、状态机驱动	Centralized	一个 Controller / FSM 容易调试
业务复杂、多个子领域协作	Delegated	主控保留决策，子领域专精
简单 CRUD、消息频繁	Delegated	避免集中式臃肿
完全 P2P 的对象交互	✗ Dispersed	控制流难追踪，慎用

Heuristics for Centralized Controller

- Avoid interaction designs where **most messages originate from a single component**.
- Keep components small.
- Make sure operational responsibilities are **not all assigned to just a few components**.
- Make sure operational responsibilities are **consistent with data responsibilities**.

PART 5 · 模块化与信息隐藏

Parnas 思想 · 内聚 / 耦合 · 详细设计的重构指南

模块化思想的历史发展

阶段	年代	代表	核心思想
萌芽	1971-72	Wirth、Parnas	逐步求精；按"职责"分解
形成	1974-85	Stevens、Parnas	耦合 / 内聚定义；信息隐藏的精细化
发展	1992-95	Eder、Hitz	OO 模块化、面向对象耦合度量
反思	1996+	McConnell、DeMarco	工程实践 / 软件度量

Good Software 关心什么？

- **Parnas 1972**: Managerial / Product flexibility / Comprehensibility
- **Stevens 1974**: 简洁性（易调试 / 易分解）、可观察性（易修改）
- **Boehm 1976**: Maintainability / Extendibility / Understandability / Reusability

Wirth 1971 · Stepwise Refinement

"Program Development by Stepwise Refinement" —— 逐步求精

- The program is gradually developed in a **sequence of refinement steps**
 - To **decompose** decisions as much as possible
 - To **untangle** aspects which are only seemingly interdependent
 - To **defer** those decisions which concern details of representation as long as possible
- Refinement of the description of **program and data structure** should proceed **in parallel**.
- The degree of modularity obtained in this way will determine the ease or difficulty with which a program **can be adapted to changes or extensions** of the purpose or changes in the environment.

"Careful programming is **not a trivial subject**." —— Wirth

Parnas 1972 · 信息隐藏的提出

"On the Criteria To Be Used in Decomposing Systems into Modules" —— 软件工程史上影响最深的论文之一

What is Modularization?

- **Module**: to be a **responsibility assignment** rather than a subprogram
- **Modularizations**: include the **design decisions which must be made before** the work on independent modules can begin ("System level" decisions)

The Criterion of Decomposition: Information Hiding

- Every module in the decomposition is characterized by its knowledge of a **design decision which it hides from all others**.
- 模块化目标：① 允许一个模块在**对其他模块代码知之甚少**的情况下编写；② 允许模块被**重新组装和替换而无需重新组装整个系统**。

Parnas · 层次结构与封装

- Hierarchical Structure
 - 模块或程序之间可定义某种关系
 - 该关系是偏序
 - 我们关心的关系是 "use" or "depends upon"
- 数据结构、其内部链接、访问过程与修改过程都是单一模块的一部分 —— 即封装（encapsulation）的思想
- 控制块的格式必须隐藏在 "control block module" 内部

模块化 ≠ 把代码切碎。模块化的本质是：让每个模块封装一个"会改变的设计决策" —— 当那个决策改变时，只需修改一个模块。

Stevens 1974 · 结构化设计

三大概念

- **Module**: A set of one or more contiguous program statements having **a name** by which other parts of the system can invoke it and preferably having its **own distinct set of variable names**
- **Coupling (耦合)**: The measure of the **strength of association** established by a connection from one module to another
- **Cohesiveness (内聚)**: 模块**内部**元素相关程度

内聚 6 等级 (弱→强)

Coincidental (偶然) → Logical (逻辑) → Temporal (时间) → Communicational (通信) → Sequential (顺序) → Functional (功能)

设计目标: 追求最强的功能内聚 (Functional Cohesion)。

内聚 / 耦合 · 代码案例

案例 A：低内聚（坏）

```
public class Rous {  
    public static int findPattern(  
        String text, String pattern) { ... }  
    public static int average(  
        Vector numbers) { ... }  
    public static OutputStream openFile(  
        String fileName) { ... }  
}
```

逻辑内聚（甚至偶然内聚）：三个方法毫无关联，硬塞在一起。

案例 B：高耦合（坏）

```
public class Employee {  
    public String name, emailID;  
    ...  
}  
public class EMailer {  
    public void sendEmail(  
        Employee e, String text) { ... }  
    ...  
}
```

数据耦合：EMailer 直接访问 Employee 的公开字段，破坏封装。改进：让 Employee 提供 `getEmailID()`，或传递 String email。

P1—P4 · 信息隐藏基础原则

编号	原则	含义
P1	全局变量有害	全局变量穿透所有模块，是隐藏的反义词
P2	优先显示表达	用清晰的接口、命名、文档 显式 表达约束，而不是依赖隐式约定
P3	不要重复（DRY）	同一个决策不应在多处编码；重复 = 多处都需修改
P4	面向接口编程	客户端依赖 接口（抽象） ，而非具体实现；便于替换

这 4 条原则全部源自一个共同目标：让**"会变的部分"**集中在一个模块里，并对外只暴露**稳定的接口**。

SWEBOK §1.4 软件设计原则全图

SWEBOK 原则	含义	本课对应原则编号
Abstraction（抽象）	聚焦必要属性，忽略其余细节	P4（接口=抽象）
SoC（关注点分离）	将系统分解为相互独立的关注点	P13 SRP；P6 ISP
Modularization（模块化）	分解为具名、接口良好的独立单元	P1—P4（基础）；P8—P9
Encapsulation（封装/信息隐藏）	隐藏设计决策，只暴露稳定接口	P1—P4；P9—P10
Separation of Interface & Impl	接口对客户可见，实现对客户不可见	P4 （面向接口）； P12 DIP
Coupling（耦合）	模块间依赖强度，越低越好	P5 LoD； P6 ISP； P8 组合
Cohesion（内聚）	模块内部元素关联程度，越高越好	P13 SRP
Uniformity（一致性）	命名、接口、约定在系统内保持一致	P14 SOFA（A — 抽象层次一致）
Completeness（完备性）	设计充分捕获抽象的所有重要特征	P11 OCP（对所有扩展点开放）
Verifiability（可验证性）	设计信息足以验证约束与需求	P7 LSP（契约可验证）

PART 6 · 为类间协作开发集成测试

Mock Object · 自顶向下 / 自底向上集成

详细设计的集成测试

- 类间协作的集成测试
 - 重点针对**复杂逻辑**（交互比较多）的协作
 - **自顶向下** 或 **自底向上** 的集成
- Mock Object
 - **不是 Stub** —— Mock 是更轻量的"占位类"
- 测试用例
 - 输入：协作中需要的对象 + 触发消息
 - 输出：验证最终状态或返回值（`assertEquals`）

协作图示意（被测目标）

```
salesController → Sales.total() → SalesList.total() → SalesLineItem.subtotal() → Commodity.getPrice()
```

Mock Object（模拟对象）

MockObject 示例

```
public class MockCommodity
    extends Commodity {
    double price;
    public MockCommodity(double p) {
        price = p;
    }
    public double getPrice() {
        return price;
    }
}
```

类似于 Stub，但**更简单**——只需返回固定值用于测试。

集成测试代码

```
public class TotalIntegrationTester {
    @Test
    public void testTotal() {
        MockCommodity c1 = new MockCommodity(50);
        MockCommodity c2 = new MockCommodity(40);
        SalesLineItem li1 = new SalesLineItem(c1, 2);
        SalesLineItem li2 = new SalesLineItem(c2, 3);

        Sales sale = new Sales();
        sale.addSalesLineItem(li1);
        sale.addSalesLineItem(li2);

        assertEquals(220, sale.total());
    }
}
```

Mock 让我们只关心被测协作的逻辑，把外部依赖替换为可控的"假对象"。

PART 7 · AI 增强：LLM 辅助详细设计

让大模型成为你的"职责分配教练"

AI 辅助 · 职责分配 Prompt 模板

你是一位资深面向对象设计教练，请帮我用 GRASP 模式分配职责。

【需求】

我在做一个 Monopoly（大富翁）游戏。每个回合玩家执行：

- (1) 掷骰子计算随机数；
- (2) 找到玩家当前位置；
- (3) 根据随机数和当前位置确定新 Square；
- (4) 将玩家移动到新 Square。

【已有概念类】

– MonopolyGame、Die、Player、Piece、Board、Square

请按以下顺序回答：

1. 用 Information Expert / Creator / Controller 三种 GRASP 分别评估每一步应分配给哪个类，给出理由。
2. 画出顺序图（PlantUML 文本格式）。
3. 评估两种候选方案的耦合 / 内聚 / 演化稳定性。
4. 推荐最终方案并说明权衡。

课堂实操：把这个 Prompt 喂给 Claude / GPT，对比 AI 答案与你的设计 —— 培养"批判性使用 AI"的能力。

AI vs 详细设计师 · 能力边界 ①：图表生成与模式识别

能力维度	LLM 表现	人类设计师优势	研究依据
单类 UML 类图生成	✅ 88% 元素正确率	—	GPT-4V, LLM4Code 2024
多类关系类图	⚠️ 语义错误显著增多	✅ 正确处理多元关系	ESEM 2024
多智能体 UML 类图 (NOMAD)	✅ 认知启发式多智能体, 优于单智能体	—	arXiv:2511.22409
多模态顺序图生成 (MM-LLM)	✅ BLEU=0.779, SSIM=0.942	—	arXiv:2503.12293
设计模式实现代码	✅ 比人类更准确	✅ 识别何时需用模式	JSS 2025
设计模式识别	⚠️ 特定模式类型存在系统性弱点	✅ 上下文感知识别	arXiv:2501.04835
耦合/内聚 (有引导)	✅ 基准正确率可接受	—	arXiv:2511.20933
耦合推理 (复杂/噪声)	❌ F1 降幅 >50%	✅ 系统性分析	arXiv:2511.20933

规律：LLM 在局部、有约束的生成任务（单类图、标准模式实现）表现良好；跨类语义推理和多重关系建模是核心弱点。

AI vs 详细设计师 · 能力边界 ②：规范遵守与代码质量

能力维度	LLM 表现	人类设计师优势	研究依据
SOLID 原则检测（小方法）	✅ OCP 71.2%；DIP 64.8%（GPT-4o Mini）	—	arXiv:2509.03093
SOLID 检测（复杂代码）	❌ 每 +5 LOC，F1 下降约 15%	✅ 复杂类体系分析	arXiv:2509.03093
遵守 SRP / 模块化	❌ 默认违反 SRP，耦合度上升 34%	✅ 识别职责边界	arXiv:2605.02741
修复简单代码异味	✅ GPT-4o 解决约 62%	—	IEEE TSE 2025
修复架构级代码异味	❌ 解决率 47.7%；63.1% 检测为误报	✅ 架构级重构	arXiv:2605.07001
库级系统设计（全任务）	❌ 完成率仅 3%；函数级 82% 降至库级 45%	✅ 全局系统设计	arXiv:2605.03546
生成架构决策记录（ADR）	⚠️ 相关但低于人类质量	✅ 决策权衡与上下文	arXiv:2403.01709
选择控制风格	⚠️ 倾向"集中式"	✅ 结合质量属性权衡	经验观察

结论：LLM 是高效的"设计初稿生成器"，但当任务涉及语义正确性、模块化约束、架构级权衡时，人类设计师仍不可缺。

LLM 辅助设计：研究进展（UML生成）

能做什么

- **GPT-4V** 单类图 **88% 元素正确率**；多类图显著下降（LLM4Code 2024, arXiv:2404.14370）
- **Rubei et al.** 评估 ChatGPT 生成顺序图：一致性尚可，需求模糊时完整性明显下降（SoSyM 2025, arXiv:2404.06371）
- **ESEM 2024**：LLM 类图与人类在可理解性相近，但**语义错误**（关系、多重度）更多
- **NOMAD**（arXiv:2511.22409, 2025）：多智能体认知架构分解类图生成子任务，**优于单智能体**
- **多模态 MM-LLM**（arXiv:2503.12293, 2026）：顺序图 BLEU=**0.779**，SSIM=**0.942**；领域自适应大幅提升精度
- **JSS 2025**：设计模式**实现**优于人类，但**识别何时需要**模式仍是人类优势

还不能做什么

多类关系与语义完整性

多个类之间的依赖、继承、组合关系是 LLM 的弱点——单个类可以处理，跨类语义经常出错。

需求质量依赖

LLM 生成质量与输入需求质量高度相关；模糊需求输入会导致图表结构性偏差。

MDD 工具链：Sadik et al. (2024) 用 PlantUML 作为 GPT-4 的形式化桥接，构建 Agile MDD 流水线（arXiv:2310.04304）。

LLM 辅助设计：代码质量 ① — 结构性缺陷

量质反比定律

代码体积几乎是结构退化的完美预测因子。

— arXiv:2605.02741, **AI-Generated Smells**, 2025

- LLM 生成代码比人类基线多 **42%—85%** 代码异味 (arXiv:2510.03029)
- 简单异味可修复：Magic Number、Empty Catch
- **复杂结构性异味无法有效修复**：Broken Modularization、Deficient Encapsulation (ACM TOSEM 2024)
- 模型能力越强 → 代码体积越大 → **结构越臃肿、耦合越高**

模块化悖论

"即使给出显式的模块化指导，LLM 仍产生含有隐藏依赖和过度工程的代码。"

— The Modular Imperative, Harvard, ICFP/SPLASH 2025

- LLM 倾向于把新功能加入已有类而非新建职责单一的类 → **默认违反 SRP**

异味修复能力对比

异味类型	LLM 能力
Magic Number / Empty Catch	✅ ~62% 解决率
Broken Modularization	❌ 显著劣于人类
Deficient Encapsulation	❌ 显著劣于人类
God Class (有工具辅助)	⚠️ iSMELL F1=75%
架构级异味 (SmellBench)	⚠️ 47.7% 解决；63.1% 误报
少样本重构	⚠️ 42.97% 减少率

结构性规律：LLM 可以修复表层语法级异味，但对涉及**职责边界和模块语义**的架构级异味束手无策。

LLM 辅助设计：代码质量 ② — 推荐工具链

工具	用途	效果	来源
iSMELL	LLM + 7 专家静态分析工具；检测 God Class、Feature Envy 等	F1 提升 +35% vs 纯 LLM	ASE 2024
SmellBench	架构级代码异味修复基准（11 智能体 × 4 模型 × 65 种异味）	最佳解决率 47.7% ；误报率 63.1%	arXiv:2605.07001
NOMAD	多智能体 UML 类图生成；认知启发式子任务分解	显著优于单智能体	arXiv:2511.22409
DRAFT	RAG + 微调；自动生成架构决策记录（ADR）	相关性优于零样本 LLM	arXiv:2504.08207
LLM-ADD	LLM 融入属性驱动设计（ADD）迭代流程	设计迭代时间缩短 40%	arXiv:2506.22688
MS Arch Agent	上传代码/YAML，自动生成架构图并检测风险	工业实践验证	Microsoft 2024
ClarifyGPT	检测模糊需求并追问澄清，再生成代码	Pass@1 提升 +10%	FSE 2024

使用建议：工具链的价值在于**组合**——用 iSMELL 检测、SmellBench 评估、NOMAD 生成初稿、ClarifyGPT 澄清需求，各工具覆盖 LLM 独立完成时的盲区。

LLM 辅助软件设计：2025—2026 最新进展

UML 生成与设计辅助

- **NOMAD** (arXiv:2511.22409, 2025): 认知启发式多智能体, 将类图生成分解为目标导向子任务, **优于单智能体方案**
- **多模态 MM-LLM** (arXiv:2503.12293, 2026): 顺序图生成 BLEU=**0.779**, SSIM=**0.942**; 领域自适应模型显著提升图→代码精度
- **LLM-ADD** (arXiv:2506.22688, 2026): 将 LLM 融入属性驱动设计 (ADD), **设计迭代时间缩短 40%**, 质量接近人类架构师
- **C4 架构多智能体** (arXiv:2510.22787, 2025): 角色专属智能体协作, C4 架构图生成准确率 **78%**
- **实践调研** (arXiv:2605.01392, 2026): 分析 291 次 ChatGPT-开发者对话, 识别 **9 类设计任务**; 68% 对话成功解答设计问题

规模化设计的新挑战

规模化崩溃 (arXiv:2605.03546)

函数级正确率 **82%** → 库级全任务完成率仅 **3%**

结构化规格说明可缓解, 提升精度 **+23%**

架构级代码异味新证据 (arXiv:2605.02741)

LLM 生成代码的"管理者类"使**耦合度平均上升 34%**; 智能体拆分文件但无法实现语义内聚

SmellBench 2026 最新基准

11 种智能体 × 4 种模型 × 65 个架构异味:
最佳解决率 **47.7%**, 但 **63.1%** 检测结果是误报
→ 人工验证仍不可省

如何正确使用 LLM 辅助详细设计

推荐 workflow

- ① 人类主导：职责边界识别 (GRASP Expert/Creator)
↓
- ② LLM 生成：类图骨架 / 顺序图初稿 (PlantUML)
↓
- ③ 人类验证：语义正确性 / 多类关系 / 耦合级别
↓
- ④ LLM 辅助：简单代码异味修复 / Javadoc / 测试生成
↓
- ⑤ 人类决策：架构级重构 / 信息隐藏决策 / 设计理由

关键 Prompt 工程技巧

给 LLM 约束，而非只给需求

请帮我设计 [模块名] 的类图。

约束条件：

- 每个类只有一个变化原因 (**P13** SRP)
- 类之间只通过接口依赖 (**P12** DIP)
- 不暴露集合类型字段 (封装B)

请先列出每个类的"主要秘密"

(隐藏的设计决策)，再画类图。

研究结论：加入设计约束的提示词

(Context-specific Prompting) 可将代码异味减少最高 **89%**，Chain-of-Thought 提示对结构性设计决策也有显著帮助

(Information and Software Technology, 2025)。

详细设计文档结构

1 引言

- 1.1 编制目的 — 文档读者与主题
- 1.2 词汇表 — 缩写、专业词汇
- 1.3 参考资料 — 相关文献

2 中层设计

2.1 xxx 模块的静态结构和动态行为

- 2.1.1 局部模块的职责（逻辑视角 / 结构视角 / 依赖视角）
- 2.1.2 局部模块的接口规范（供接口 / 需接口）
- 2.1.3 模块的行为（顺序图 / 通讯图 / 状态图）
- 2.1.4 模块的实现注解（构造方法、枚举、常量、静态方法）
- 2.1.5 业务逻辑模块的设计原理（Rationale）

3 低层设计（如需要）

3.1 关键数据结构与算法

详细设计的评审（16 项检查清单）

一、基本

1. 设计方案自身是否一致？ 2) 设计制品的详细程度是否合适？ 3) 设计是否包含了各个视角？ 4) 多个视角之间是否一致？

二、设计考量

5. 是否采用标准技术？ 6) 是否强调简洁性重于灵活性？ 7) 是否尽可能简单？ 8) 是否精干？每个部分都必需？ 9) 维护时需求变更是否支持？是否支持未来的扩展？ 10) 是否支持重用？ 11) 是否具有低复杂性？ 12) 是否可理解？

三、过程考量

13. 是否覆盖了所有需求？ 14) 设计的功能对应需求的哪些部分？ 15) 是否足够遵循软件体系结构的设计决策？ 16) 详细程度对后继开发人员是否足够？

详细设计的验证

评审 (Review)

- 团队走查 (Walkthrough)
- 检查清单 (Checklist)
- 形式化评审 (Inspection)

度量 (Metrics)

- 模块化度量
 - 耦合度 C_a / C_e
 - 不稳定性 $I = C_e / (C_a + C_e)$
 - 抽象度 A
 - 主序列距离 $D = |A + I - 1|$

测试

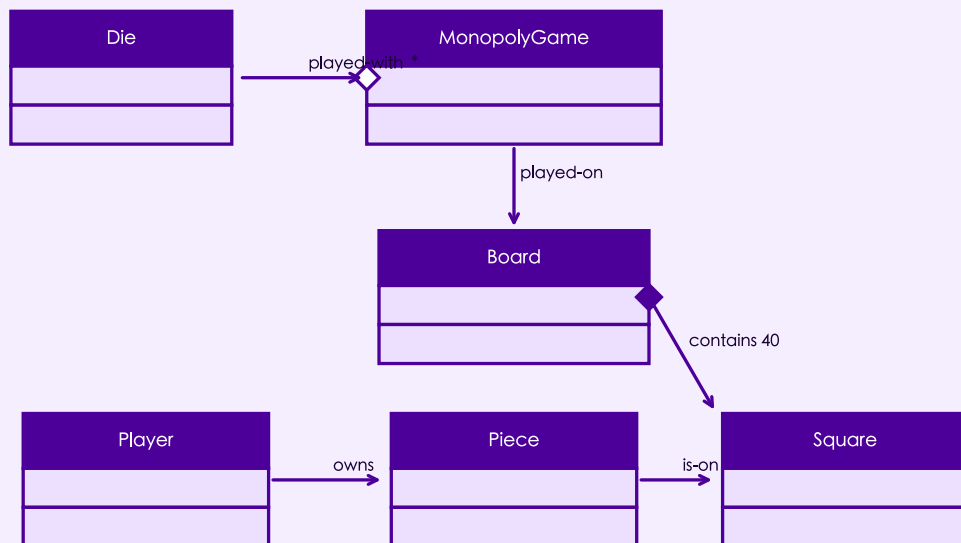
- 协作测试 (Class Collaboration Test)
- Mock-Driven 测试
- 变化测试 (Mutation Testing)

评审的常见缺陷

- 多视角不一致 (类图 vs 顺序图)
- 接口规格不完整
- 缺失实现注解
- 设计理由 (Rationale) 缺失

课堂练习 · Monopoly Game

已知类图



题目

1. 掷骰子的行为应由哪个类执行? —— **Die** (信息专家)
2. 根据 name 寻找相应的 Square? —— **Board** (聚合 Square)
3. 回合调度应由谁? —— **MonopolyGame** (Controller)
4. 整个执行过程的调度者? —— **MonopolyGame**
5. 持久化 / 存档? —— **新建 IO/Persistence 类** (Pure Fabrication)

课堂练习 · switch-case 反模式

```
class Square {  
    void landOn(Player p) {  
        switch (this.Type) {  
            case Tax:    收税;    break;  
            case Income: 增加收入; break;  
            case Pay:    罚款;    break;  
            ...  
            default: 没有行为;  
        }  
    }  
}
```

问题：所有 Square 类型的行为都耦合在一个 switch 语句里 —— 新增一种 Square 要修改 `landOn`，违反 P11 开闭原则（OCP）。

正解：建立行为继承树，用多态代替 switch-case —— `Tax extends Square`、`Income extends Square`, 各自实现 `landOn(Player)`。

本讲总结

关键概念

- 详细设计 = 中层 + 低层
- 职责 + 协作 = OO 详设两大支柱
- **GRASP**: Information Expert / Creator / Controller / Low Coupling / High Cohesion
- **控制风格**: 集中 / 委托 / 分散
- **P1-P4** (信息隐藏基础): 全局变量有害 / 优先显示表达 / DRY / 面向接口编程
- **Parnas 信息隐藏**: 每个模块封装一个"会变的设计决策" (→ P9 为变更而封装)
- **集成测试 + Mock** 验证协作

思维框架

1. **从需求和体系结构开始** —— 把职责落到类
2. **静态设计** —— 类、关系、接口
3. **动态设计** —— 顺序图、状态图
4. **重构** —— 模块化、信息隐藏、设计模式
5. **验证** —— 评审、度量、集成测试

课后作业

必做

1. 为大作业的**核心模块**（如订单模块）画出**类图 + 顺序图**，标注 GRASP 模式选择理由。
2. 对该模块中的一个**复杂方法**（如 `pay(orderId, couponId)`）画出**完整顺序图**，识别其使用的**控制风格**。
3. 用 **MockObject** 为该方法编写**至少 3 个集成测试用例**（包含正常路径、异常路径、边界情况）。

选做（加分）

4. 对你的设计应用**16 项评审清单**自评，写一份**评审报告**（每条给出"通过 / 不通过 / 改进建议"）。
5. 用 LLM 生成同一模块的另一份设计方案，**对比两份方案的耦合 / 内聚 / 演化稳定性**，写出**批判性分析**（300 字以内）。

Q & A

详细设计 · 把架构落到代码的最后一公里

南京大学 · 刘钦 · 软件工程与计算 II